Curso
Tendencias Aplicadas de Ingenería de Software

Persistencia de Datos No Relacional
Bases de datos más comunes
Tácticas Posibles de Persistencias

Clasificación General de las Bases NoSQL

- Aunque frecuentemente se habla de NoSQL como una única categoría, en realidad se trata de una familia de tecnologías con modelos de datos muy diferentes.
- Las cuatro categorías más relevantes son:
- ***Bases de datos Key-Value***
  - Optimizadas para accesos extremadamente rápidos mediante claves únicas.
- ***Bases documentales***
  - Orientadas al almacenamiento de documentos JSON o estructuras semiestructuradas.
- ***Bases Column-Family***
  - Diseñadas para manejar enormes volúmenes de datos distribuidos.

Clasificación General de las Bases NoSQL

- ***Bases de grafos***
  - Especializadas en representar relaciones complejas entre entidades.
- Cada modelo responde a necesidades arquitectónicas distintas.
- Por este motivo, uno de los errores más comunes consiste en seleccionar una base NoSQL únicamente por popularidad sin analizar previamente los patrones de acceso y los requisitos de negocio.
- La comprensión profunda de estos modelos permite aplicar el principio de "usar la herramienta adecuada para el problema adecuado".

Bases de Datos Key-Value

- Las bases de datos Key-Value representan la forma más sencilla de almacenamiento distribuido.
- Su estructura se basa en una asociación directa:
  - Clave → Valor
- La clave funciona como identificador único y permite recuperar el valor asociado de manera extremadamente eficiente.
- Ejemplo:
  - Usuario:123 → Datos del usuario
  - Producto:456 → Datos del producto
  - Sesion:789 → Información de sesión

Bases de Datos Key-Value

- Conceptualmente se asemejan a una tabla hash distribuida.
- La simplicidad del modelo permite optimizaciones en términos de rendimiento y escalabilidad.
- En muchos casos, el sistema no necesita comprender la estructura interna del valor almacenado, limitándose a recuperarlo utilizando la clave correspondiente.
- Esta característica convierte a las bases Key-Value en una excelente opción para aplicaciones que requieren tiempos de respuesta extremadamente bajos.

Ventajas y Desventajas de Key-Value

- Ventajas, las bases Key-Value destacan por:
  - Simplicidad conceptual.
  - Excelente rendimiento.
  - Escalabilidad horizontal sencilla.
  - Baja latencia.
  - Alta disponibilidad.
  - Implementación eficiente de cachés distribuidas.

- Muchas aplicaciones pueden procesar millones de operaciones por segundo utilizando este modelo.

Características, Ventajas y Desventajas de Key-Value

- Desventajas, no obstante, presentan limitaciones importantes:
  - Ausencia de relaciones complejas.
  - Dificultad para realizar análisis ad hoc.
  - Poca flexibilidad para búsquedas avanzadas.

- Por ejemplo, si un sistema almacena usuarios mediante identificadores únicos, recuperar un usuario por correo electrónico podría requerir estructuras adicionales.

- Estas restricciones hacen que las bases Key-Value sean ideales para algunos problemas muy específicos, pero poco adecuadas para aplicaciones analíticas complejas.

Sistemas Key-Value Representativos

- Redis es probablemente la base Key-Value más conocida del mercado.

- Sus características incluyen:
  - Almacenamiento en memoria.
  - Replicación.
  - Persistencia opcional.
  - Estructuras de datos avanzadas.
  - Publicación y suscripción.

Sistemas Key-Value Representativos

- Se utiliza frecuentemente como:
  - Caché distribuida.
  - Gestor de sesiones.
  - Cola de mensajes.
  - Almacenamiento temporal.

Bases de Datos Documentales

- Las bases documentales almacenan información en forma de documentos autónomos, generalmente utilizando formatos como JSON, BSON o XML.
- A diferencia del modelo relacional, los datos relacionados pueden almacenarse juntos dentro del mismo documento.
- Ejemplo:

```
{
        "cliente":"Juan",
        "direccion": {
                "ciudad":"Montevideo",
                "pais":"Uruguay"
         }
}
```

Bases de Datos Documentales

- Este enfoque se alinea naturalmente con los objetos utilizados en los lenguajes de programación modernos.
- Como resultado, se reduce el fenómeno conocido como "Object-Relational Impedance Mismatch", que históricamente motivó la aparición de herramientas ORM.
- Desde la perspectiva de la ingeniería de software, las bases documentales facilitan el desarrollo de aplicaciones ágiles y favorecen la evolución rápida de modelos de negocio.

Ventajas de las Bases Documentales

- ***Flexibilidad y adaptación al dominio***
  - Las bases documentales se han convertido en una de las categorías más populares dentro del universo NoSQL.

- Entre sus principales ventajas se encuentran:

  Flexibilidad de esquema
  Los documentos pueden evolucionar sin requerir migraciones masivas.

  Modelado cercano al dominio
  La estructura de almacenamiento suele reflejar directamente las entidades utilizadas por la aplicación.

Ventajas de las Bases Documentales

- **Menor necesidad de JOINs**
  La información relacionada puede almacenarse dentro del mismo documento.

- **Escalabilidad horizontal**
  Muchas plataformas documentales incorporan particionamiento distribuido nativo.

- **Desarrollo más ágil**
  La evolución funcional del sistema suele requerir menos cambios en la infraestructura de persistencia.

Desventajas de las Bases Documentales

- Asimismo, ciertas consultas analíticas complejas pueden resultar menos eficientes que en sistemas relacionales optimizados.

- Por esta razón, el diseño documental requiere comprender cuidadosamente los patrones de acceso y las necesidades de consulta del sistema.

Desventajas de las Bases Documentales

- A pesar de sus ventajas, las bases documentales presentan algunas limitaciones importantes.
- La más relevante es la tendencia a la desnormalización.
- Cuando una misma información aparece replicada en múltiples documentos:
  - Aumenta el consumo de almacenamiento.
  - Se incrementa la complejidad de actualización.
  - Pueden aparecer inconsistencias.
- Otro desafío importante es la integridad referencial.
- Mientras que en una base relacional las claves foráneas son gestionadas automáticamente por el motor, en muchas bases documentales estas validaciones deben implementarse a nivel de aplicación.

Bases de Datos Column-Family

- Las bases de datos Column-Family (también denominadas Wide Column Stores) fueron diseñadas para resolver problemas asociados al almacenamiento y procesamiento de volúmenes masivos de información distribuidos en múltiples servidores.

- A diferencia del modelo relacional, donde las filas poseen una estructura fija, en una base Column-Family cada registro puede contener diferentes conjuntos de columnas.

Bases de Datos Column-Family

- La organización física de los datos está optimizada para:
  - Escrituras masivas.
  - Consultas por rangos.
  - Procesamiento distribuido.
  - Escalabilidad horizontal.

- Este modelo resulta especialmente adecuado para aplicaciones que generan enormes cantidades de eventos, métricas o registros temporales.

- Desde una perspectiva arquitectónica, las bases Column-Family se orientan a maximizar el rendimiento.

Estructura y Modelo de Datos Column-Family

- En este modelo los datos se estructuran utilizando varios niveles de organización.
- **Clave de fila (Row Key)**
  - Identifica de manera única cada registro.
- **Familias de columnas**
  - Agrupan columnas relacionadas funcionalmente.
- **Columnas**
  - Representan atributos específicos asociados a cada fila.
- **Timestamp**
  - Muchas implementaciones almacenan múltiples versiones temporales de un mismo dato.

Estructura y Modelo de Datos Column-Family

- Una diferencia importante respecto a los modelos relacionales es que las columnas pueden variar entre registros.
- Esta flexibilidad permite optimizar el almacenamiento y reducir espacio desperdiciado cuando los datos presentan estructuras heterogéneas.
- Sin embargo, el diseño del esquema requiere una comprensión profunda de los patrones de acceso, ya que el rendimiento depende significativamente de cómo se organizan las claves y las familias de columnas.

Ventajas y Desventajas de las Bases Column-Family

- Ventajas, Las bases Column-Family destacan por:
  - Escalabilidad masiva
  - Pueden distribuir datos entre cientos o miles de nodos.
  - Excelente rendimiento de escritura
  - Son capaces de procesar enormes volúmenes de inserciones.
  - Alta disponibilidad
  - La replicación distribuida permite tolerar fallos de hardware.

Ventajas y Desventajas de las Bases Column-Family

- Desventajas
  - Diseño más complejo
  - La modelización requiere comprender los patrones de consulta desde el inicio.
  - Menor flexibilidad para consultas ad hoc
  - Las consultas no previstas pueden resultar costosas.
  - Curva de aprendizaje elevada
- El diseño efectivo exige conocimientos avanzados de sistemas distribuidos.
- Por esta razón, suelen utilizarse en proyectos de gran escala donde las ventajas justifican la complejidad adicional.

Bases de Datos de Grafos

- Las bases de datos de grafos representan una aproximación completamente diferente al almacenamiento de información.
- Mientras que los modelos relacionales y documentales se enfocan principalmente en entidades y atributos, las bases de grafos consideran que las relaciones constituyen elementos de primera clase.
- Un grafo está compuesto por:
  - **Nodos**: que representan entidades.
- Ejemplos:
  - Personas.
  - Empresas.
  - Productos.
  - Documentos.
  - Relaciones

Bases de Datos de Grafos

- **Relaciones**: conectan nodos, por ejemplo:
  - AMIGO_DE
  - COMPRA
  - SIGUE_A
  - TRABAJA_EN
  - Propiedades
- **Propiedades**:Información asociada tanto a nodos como a relaciones.
- Este modelo resulta especialmente eficaz cuando las consultas involucran múltiples niveles de relaciones complejas.

¿Por qué utilizar una base de grafos?

- Muchas aplicaciones modernas requieren explorar relaciones profundas entre entidades.
- Por ejemplo:
  - ¿Qué amigos de mis amigos compraron este producto?
  - ¿Cuál es el camino más corto entre dos personas?
  - ¿Qué dependencias existen entre microservicios?
- En bases relacionales estas consultas suelen requerir múltiples JOINs recursivos, cuyo costo crece rápidamente.

¿Por qué utilizar una base de grafos?

- Las bases de grafos almacenan explícitamente las relaciones, permitiendo recorrerlas de forma eficiente.
- Los modelos de grafos suelen ser valiosos cuando las conexiones entre entidades son tan importantes como los datos mismos.
- Por este motivo se han convertido en una tecnología fundamental en áreas como análisis social, logística y sistemas de recomendación.
- Ejemplo: Neo4j

Táctica Arquitectónica: Desnormalización

- Uno de los principios más importantes del diseño NoSQL es la desnormalización.
- En bases relacionales, la normalización busca eliminar redundancias y evitar inconsistencias.
- Sin embargo, en sistemas distribuidos las operaciones JOIN pueden convertirse en cuellos de botella significativos.
- Por este motivo, muchas soluciones NoSQL optan por almacenar datos redundantes.
- Ejemplo:

```
{
        "cliente":"Juan",
        "ciudad":"Montevideo",
        "pais":"Uruguay"
}
```

Táctica Arquitectónica: Denormalización

- Aunque la ciudad y el país existan en otros registros, se almacenan nuevamente para simplificar las consultas.
- Beneficios:
  - Menor cantidad de accesos.
  - Consultas más rápidas.
  - Menor dependencia de operaciones distribuidas.
- Costo:
  - Mayor duplicación.
  - Actualizaciones más complejas.
- La decisión debe basarse en los patrones reales de lectura y escritura del sistema.

Táctica Arquitectónica: Sharding

- El sharding consiste en dividir los datos entre múltiples nodos independientes.
- Por ejemplo:
  - Shard 1 → Clientes A–F
  - Shard 2 → Clientes G–M
  - Shard 3 → Clientes N–Z
- Cada shard almacena sólo una parte del conjunto total de datos.
- Beneficios:
  - Escalabilidad horizontal
  - Permite agregar nuevos nodos a medida que aumenta la carga.

Táctica Arquitectónica: Sharding

- Distribución de recursos
  - Reduce la presión sobre un único servidor.
  - Mayor capacidad de almacenamiento
- El sistema puede crecer prácticamente sin límites teóricos.
- No obstante, seleccionar incorrectamente la clave de particionamiento puede generar problemas de desbalanceo y afectar significativamente el rendimiento.

Táctica Arquitectónica: Replicación

- La replicación consiste en mantener múltiples copias de los mismos datos en diferentes nodos.

- Objetivos principales:
  - Alta disponibilidad
  - Si un nodo falla, otro puede continuar prestando servicio.
  - Recuperación ante desastres
  - Reduce riesgos de pérdida de información.
  - Balanceo de lectura
  - Las consultas pueden distribuirse entre varias réplicas.

Táctica Arquitectónica: Replicación

- Existen diversos enfoques:
- **Replicación síncrona**
  - Garantiza consistencia fuerte.
- **Replicación asíncrona**
  - Prioriza disponibilidad y rendimiento.
- La mayoría de las bases NoSQL utilizan estrategias híbridas para equilibrar consistencia, rendimiento y disponibilidad.
- La replicación es uno de los mecanismos que permiten construir sistemas resilientes a gran escala.

Táctica Arquitectónica: CQRS (Command Query Responsibility Segregation)

- CQRS (Command Query Responsibility Segregation) es un patrón arquitectónico que propone separar explícitamente las operaciones que modifican datos de aquellas que los consultan.
- En un sistema tradicional, una misma estructura de datos suele utilizarse tanto para lectura como para escritura. Sin embargo, los requisitos de ambos tipos de operaciones suelen ser diferentes.

Táctica Arquitectónica: CQRS (Command Query Responsibility Segregation)

- Las escrituras generalmente requieren:
  - Validaciones de negocio.
  - Control transaccional.
  - Consistencia.
  - Reglas de integridad.
- Las lecturas suelen priorizar:
  - Rapidez.
  - Escalabilidad.
  - Consultas complejas.
  - Agregaciones.
-

Táctica Arquitectónica: CQRS (Command Query Responsibility Segregation)

- CQRS propone construir modelos distintos para cada necesidad.
  - Modelo de Comandos Responsable de:
    - Crear datos.
    - Modificar datos.
    - Eliminar datos.
  - Modelo de Consultas, pptimizado para:
    - Búsquedas.
    - Reportes.
    - Dashboards.
    - Consultas analíticas.

Táctica Arquitectónica: CQRS (Command Query Responsibility Segregation)

- Esta separación permite utilizar tecnologías diferentes para cada propósito.
- Por ejemplo:
  - Escrituras → PostgreSQL
  - Lecturas → MongoDB

  u otro ejemplo:

  - Escrituras → SQL Server
  - Consultas → Elasticsearch

Persistencia Políglota (Polyglot Persistence)

- Utilizando múltiples tecnologías de persistencia
- Uno de los conceptos más importantes de la arquitectura moderna es que no existe una única tecnología de persistencia adecuada para todos los problemas.
- Polyglot Persistence propone seleccionar la base de datos más adecuada para cada necesidad específica.

Persistencia Políglota (Polyglot Persistence)

- Cada componente utiliza la tecnología que mejor se adapta a sus requisitos.
- Esta estrategia resulta especialmente frecuente en arquitecturas de microservicios, donde cada servicio puede gestionar su propia persistencia.
- La persistencia políglota representa una evolución natural respecto al paradigma tradicional de una única base de datos corporativa.

Consistencia en Sistemas NoSQL

- Uno de los cambios conceptuales más importantes al trabajar con NoSQL es comprender que la consistencia puede definirse de múltiples maneras.
- En sistemas relacionales clásicos predominan las propiedades ACID:
  - Atomicity
    Las operaciones se ejecutan completamente o no se ejecutan.
  - Consistency
    Las restricciones de integridad siempre se mantienen.
  - Isolation
    Las transacciones concurrentes no interfieren incorrectamente.
  - Durability
    Los cambios persisten incluso ante fallas.

Consistencia en Sistemas NoSQL

- Muchos sistemas NoSQL alivianan algunos de estos requisitos para obtener:
  - Mejor rendimiento.
  - Mayor disponibilidad.
  - Escalabilidad horizontal.

- La consistencia deja de ser una propiedad absoluta para convertirse en un conjunto de compromisos cuidadosamente diseñados.

NoSQL y Microservicios

- Una combinación frecuente, la adopción de microservicios impulsó significativamente el crecimiento de las bases NoSQL.

- Cada microservicio:
  - Posee responsabilidades específicas.
  - Gestiona su propio ciclo de vida.
  - Mantiene su propia persistencia.

- Esto permite seleccionar tecnologías distintas para cada contexto.

NoSQL y Microservicios

- Ejemplo:
  - Usuarios → PostgreSQL
  - Catálogo → MongoDB
  - Caché → Redis
  - Recomendaciones → Neo4j
- Esta independencia favorece:
  - Escalabilidad.
  - Despliegues autónomos.
  - Evolución tecnológica.
- La persistencia políglota se convierte así en una consecuencia natural de las arquitecturas basadas en microservicios.

Conclusiones

- Las bases de datos NoSQL surgieron como respuesta a los desafíos de escalabilidad, disponibilidad y distribución que caracterizan a las aplicaciones modernas.
- NoSQL no reemplaza completamente al modelo relacional.
- Existen múltiples familias de bases de datos con objetivos diferentes.
- Los sistemas distribuidos obligan a realizar compromisos entre consistencia y disponibilidad.
- La elección de la persistencia constituye una decisión arquitectónica estratégica.

Conclusiones

- Tácticas como sharding, replicación, CQRS permiten construir sistemas altamente escalables.
- La persistencia políglota representa una tendencia dominante en arquitecturas modernas.
- Finalmente, una de las enseñanzas más importantes de Kleppmann y Sommerville es que el éxito de una arquitectura no depende de la tecnología elegida, sino de qué tan bien dicha tecnología satisface los requisitos funcionales y no funcionales del sistema que se está construyendo.